NED UNIVERSITY

DEPARTMENT OF MECHANICAL ENGINEERING

# OPEN ENDED LAB

## (REGRESSION ANALYSIS)

ME-214

Computer Programming & Applications

First Year

Batch 2024

Spring Semester-2025

Syeda Umayma Danish

ME 24008

Section A

Course Instructor: Miss Bushra Fatima

# REGRESSION ANALYSIS REPORT:

## WIND SPEED VS. POWER

## OBJECTIVE:

The purpose of this analysis was to develop an accurate mathematical model for a mechanical engineering-related dataset obtained from Kaggle and identify the best-fitting model based on R-squared and RMSE metrics.

**Constraints:**

- Use MATLAB programming language
- Use mechanical engineering related dataset obtained from Kaggle.
- Use only one independent variable and one dependent variable.
- Use only the following model types(excluding linear regression):

1. Polynomial (degree 2 - 4)
2. Logarithmic ($y = m \cdot \ln(x)+b$)
3. Exponential ($y = b \cdot e^{m \cdot x}$)
4. Power ($y = b \cdot x^m$)
5. Reciprocal ($y = 1/(m \cdot x+b)$)

## DATA:

**Name:** Wind Power Generation Data - Forecasting (Location1)
**Source:** kaggle (https://www.kaggle.com/datasets/mubashirrahim/wind-power-generation-data-forecasting?select=Location1.csv)
**Description:**
This dataset is a compilation of wind power generation data, collected directly from one company's wind turbines operational sites. This dataset represents a detailed hourly record, starting from January 2, 2017. It provides real-world insights into the interplay between various weather conditions and wind energy production.
It contains data obtained from 4 different locations. Data from location 1 is used for analysis in this report.

**The columns in the data are as follows:**
**Time** - Hour of the day when readings occurred
**temperature_2m** - Temperature in degrees Fahrenheit at 2 meters above the surface
**relativehumidity_2m** - Relative humidity (as a percentage) at 2 meters above the surface
**dewpoint_2m** - Dew point in degrees Fahrenheit at 2 meters above the surface
**windspeed_10m** - Wind speed in meters per second at 10 meters above the surface
**windspeed_100m** - Wind speed in meters per second at 100 meters above the surface
**winddirection_10m** - Wind direction in degrees (0-360) at 10 meters above the surface
**winddirection_100m** - Wind direction in degrees (0-360) at 100 meters above the surface
**windgusts_10m** - Wind gusts in meters per second at 100 meters above the surface
**Power** - Turbine output, normalized to be between 0 and 1 (i.e., a percentage of maximum potential output)

**Choosing independent and dependent variables:**
**x (independent) :** windspeed_100m
**y(dependent) :** Power
The columns windspeed_100m and power are chosen as independent and dependent variables respectively to find the model that best describes how a change in windspeed at 100m effects the power generated by the wind turbine.

## USING MATLAB TO ANALYZE THE DATA

**Importing data into MATLAB:**
Data was successfully imported into MATLAB using 'Import data' function. The reason for displaying the first five rows of data was to ensure that correct data was imported into MATLAB.

```
% Importing Data
% Import data from text file
data = readtimetable("D:\Downloads\Location1.csv", "RowTimes", "Time");
% Display first 5 rows
data([1:5],:)
```

```
ans = 5×9 timetable
```

| | Time | temperature_2m | relativehumidity_2m | dewpoint_2m |
|---|---|---|---|---|
| 1 | 2017-01-02 00:00:00 | 28.5000 | 85 | 24.5000 |
| 2 | 2017-01-02 01:00:00 | 28.4000 | 86 | 24.7000 |
| 3 | 2017-01-02 02:00:00 | 26.8000 | 91 | 24.5000 |
| 4 | 2017-01-02 03:00:00 | 27.4000 | 88 | 24.3000 |
| 5 | 2017-01-02 04:00:00 | 27.3000 | 88 | 24.1000 |

**Checking size of data:**
Analyzing the data's dimensions to grasp the overall size of the dataset.

```
% --- Data Extraction and Initial Checks ---
% Size of data
fprintf('Initial data size: %d rows, %d columns\n', size(data));
```

```
Initial data size: 43800 rows, 9 columns
```

**Extracting the chosen columns:**
We first checked whether the columns we chose (windspeed_100m and power) existed in the dataset or if their was some spelling or grammatical mistake (e.g. Capitalizing some letters). This step ensures the columns exist and we are using the correct name of the columns.

```
% Ensure column names match your data table
if ~ismember('windspeed_100m', data.Properties.VariableNames)
    error('Column ''windspeed_100m'' not found in the data table.');
end
if ~ismember('Power', data.Properties.VariableNames)
     error('Column ''Power'' not found in the data table.');
end

% Extracting the columns
x = data.windspeed_100m; % Independent variable: Wind Speed at 100m
y = data.Power;          % Dependent variable: Power
```

**Checking data for negative or zero values:**
This step is crucial for models requiring positive 'Power' values (Logarithmic, Power, Exponential) and non-zero 'Power' values (Reciprocal).

```
% Checking for negative values in y
y_negative = y(y < 0);
if ~isempty(y_negative)
    warning('Found %d negative values in y (Power). These rows will be removed.\n',
length(y_negative));
else
    disp('No negative values found in y (Power)');
end
```

```
No negative values found in y (Power)
```

```
% Checking for zero values in y
y_zero = y(y == 0);
if ~isempty(y_zero)
    warning('Found %d zero values in y (Power). These rows will be removed.\n',
length(y_zero));
else
    disp('No zero values found in y (Power)');
end
```

```
Warning: Found 101 zero values in y (Power). These rows will be removed.
```

No negative values were found but there were 101 zero values in the power column.
It's possible for a windmill to generate zero power even with some wind speed. This can happen if the wind speed is below the windmill's cut-in speed (which is the minimum wind speed below which they won't begin to turn and generate power), if the blades are frozen solid, or if there's a complete mechanical or electrical failure. The power output of a windmill is also influenced by the turbine's design, the efficiency of the components, and the overall wind conditions.
As only 101 values were zero compared to the 43800 rows, this is likely an error in data entering so for simplicity we are removing the rows with zero values.

**Cleaning data:**

```
% --- Data Cleaning ---
% Removing NaNs first
valid_nan = ~isnan(x) & ~isnan(y);
x = x(valid_nan);
y = y(valid_nan);
fprintf('Removed %d rows with NaN values.\n', size(data, 1) - length(y));
```

```
Removed 0 rows with NaN values.
```

```
% Removing negative and zero values of y
% This ensures y is positive for log and non-zero for reciprocal transformations
valid_y = (y > 0);
x = x(valid_y);
y = y(valid_y);

fprintf('Removed %d rows with non-positive Power values.\n', length(valid_nan) -
length(y));
```

```
Removed 101 rows with non-positive Power values.
```

```
% Remaining data points after cleaning
fprintf('Remaining data points: %d\n', length(y));
```

```
Remaining data points: 43699
```

0 NaN values, 0 negative values and 101 zero values were removed. The remaining rows are 43699.

**Preparing data for Plotting:**

Using linspace function to increase the number of points to plot which makes the resulting plot significantly smoother.

```
% --- Data Preparation for Plotting ---
% Expanding x for smooth plotting
x_plot = linspace(min(x), max(x), 500);
```

**Function for calculating error:**

A function was generated as the values for $r^2$ and rmse will need to be calculated multiple times, and functions help in automating repetitive tasks.

**R-squared**, also known as the coefficient of determination tells you how well the model "fits" the observed data. R-squared values range from 0 to 1 (or 0% to 100%). A higher R-squared value (closer to 1) means a better fit.

$$R^2 = 1 - \frac{\sum(y_i - \hat{y})^2}{\sum(y_i - \bar{y})^2}$$

Where,

$\hat{y}$ – *predicted value of y*

$\bar{y}$ – *mean value of y*

**RMSE (Root Mean Squared Error)** measures the average magnitude of the errors in a model's predictions. A lower RMSE value indicates that the model's predictions are closer to the actual values, suggesting a better fit and more accurate predictions.

$$RMSE = \sqrt{\sum_{i=1}^{n} \frac{(\hat{y}_i - y_i)^2}{n}}$$

$\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_n$ are predicted values

$y_1, y_2, \ldots, y_n$ are observed values

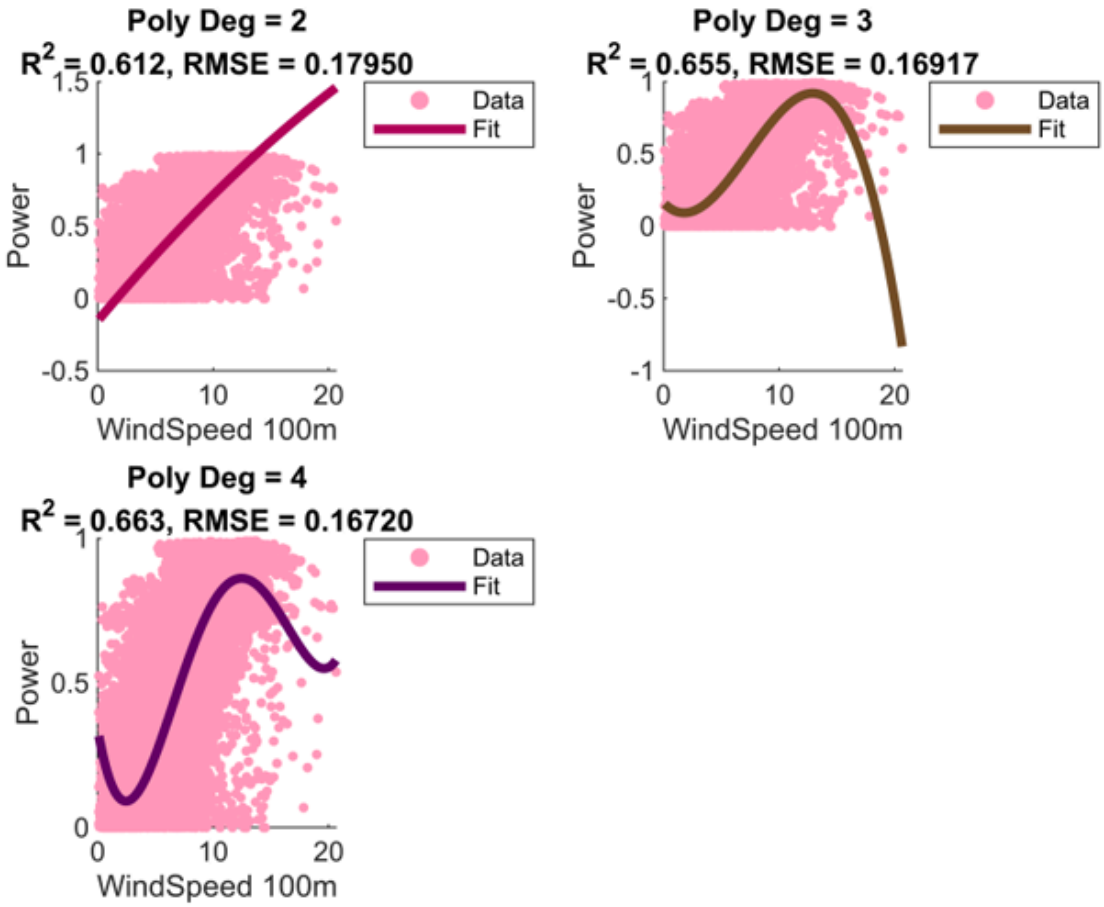
$n$ is the number of observations

```
% --- Function to calculate R-squared and RMSE ---
function [r_squared, rmse] = error(y, y_pred)
    % Check that y and y_pred have the same length before proceeding
    if length(y) ~= length(y_pred)
        warning('Input vectors for error calculation have different lengths.');
        r_squared = NaN;
        rmse = NaN;
        return;
    end
% Calculate R-squared
r_squared = 1 - (sum((y - y_pred).^2) / sum((y - mean(y)).^2));

% Calculate RMSE
rmse = sqrt(mean((y - y_pred).^2));
end
```

## Models and Plotting:

**Polynomial Models (Degrees 2, 3, and 4):** These models fit a curved line to the data. A polynomial of degree n has the form $\mathbf{y = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x^1 + a_0}$. Higher degrees can capture more complex curves but may also overfit the data. We tested degrees 2 (quadratic), 3 (cubic), and 4 (quartic).

```
% --- Regression Models and Plotting ---

% -- Polynomial Model (Degrees 2 to 4) --
poly_degrees = 2:4;
r2_poly = zeros(size(poly_degrees));
rmse_poly = zeros(size(poly_degrees));
poly_y_plots = cell(size(poly_degrees)); % To store the plot data for each degree
colors_poly = {'#b30059', '#734d26', '#660066'}; % Define colors for different
polynomial degrees

figure('Name','Polynomial Fits(Degree 2 to 4)','NumberTitle','off');

for i = 1:length(poly_degrees)
    n = poly_degrees(i);
    p = polyfit(x,y,n);
    y_poly = polyval(p,x);
    y_poly_plot = polyval(p,x_plot);

    [r2_poly(i),rmse_poly(i)] = error(y,y_poly);

    poly_y_plots{i} = y_poly_plot; % Store the plot data
    subplot(2,2,i);
    scatter(x,y,10,'o','filled','MarkerFaceColor','#ff99bb');
    hold on;
    plot(x_plot,y_poly_plot,'Color',colors_poly{i},'LineWidth',3);
    title(sprintf('Poly Deg = %d\nR^2 = %.3f, RMSE =
%.5f',n,r2_poly(i),rmse_poly(i)));
    xlabel('WindSpeed 100m');
    ylabel('Power');
    legend('Data','Fit','Location','northeastoutside');
end
```




**Graph:**
This figure contains multiple subplots, one for each polynomial degree tested. Each subplot shows the original data points (scatter plot) and the fitted polynomial curve. You can visually inspect how well each polynomial degree captures the trend in the data. The title of each subplot includes the specific R-squared and RMSE for that polynomial degree.

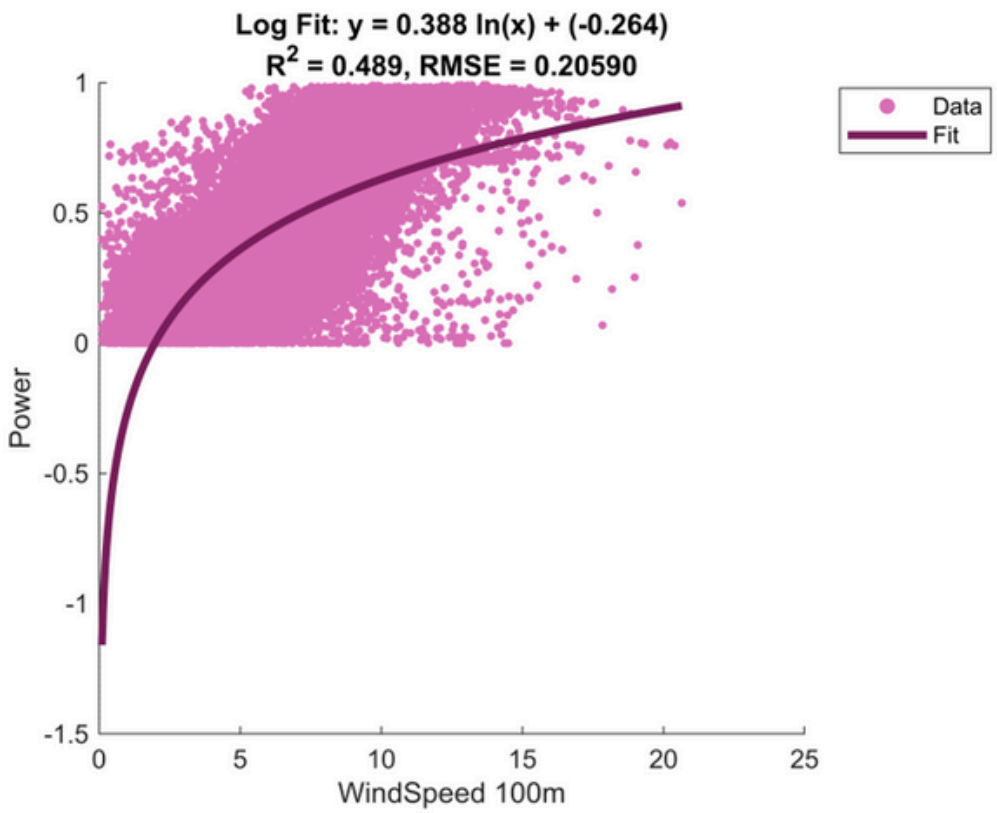
<u>Degree 2 Polynomial:</u>
The graph shows a smooth parabolic curve overlaid on the scatter plot of data points. The curve captures the general upward trend but fails to model the sharp rise in power output at higher wind speeds. It underfits the data, especially in the mid-to-high range and has limited flexibility.

<u>Degree 3 Polynomial:</u>
It adds curvature flexibility with a cubic shape and has slight improvement in fit, especially at mid-range wind speeds. However, slight deviations still appear at the extremes. It is a better alternative to degree 2, but still not optimal.

<u>Degree 4 Polynomial:</u>
The curve closely hugs the data points across the entire range and has excellent fit across all wind speed ranges, capturing the nonlinearity and sharp rise in power generation at higher speeds. It is the most accurate visual representation of the data yet.

**Logarithmic Model:** This model assumes a relationship where the dependent variable (y) changes linearly with the natural logarithm of the independent variable (ln(x)). Its form is **y=m · ln(x)+b**.

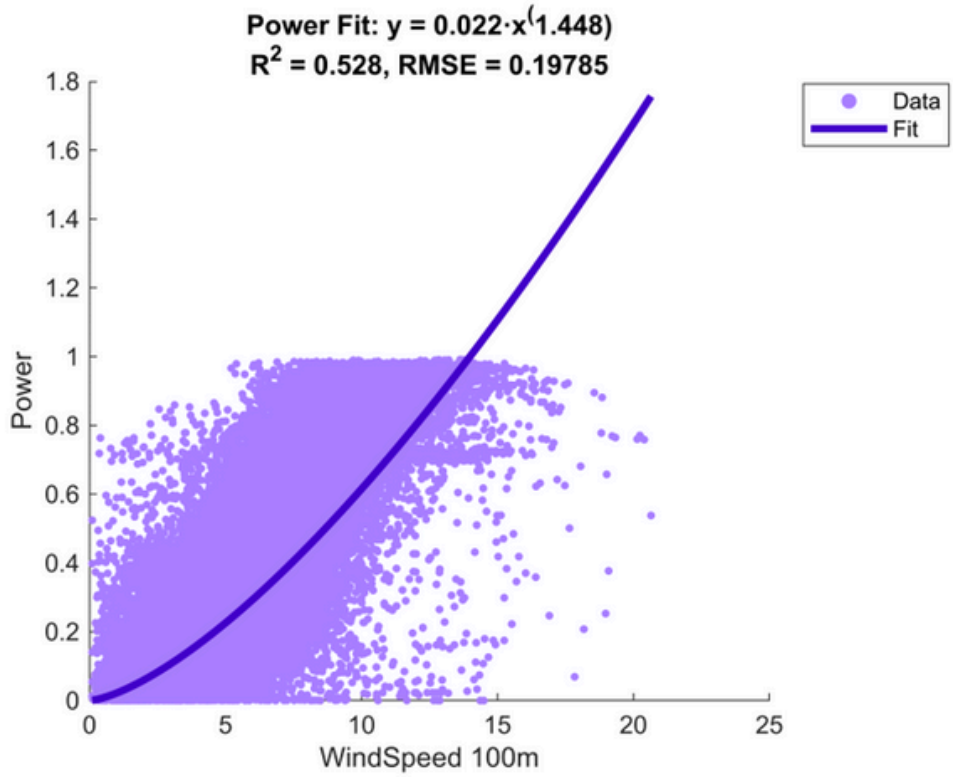
```
% -- Logarithmic Model: y = m*ln(x)+b --
lnx = log(x);
p_log = polyfit(lnx,y,1);
m_log = p_log(1);
b_log = p_log(2);
y_log = m_log*lnx+b_log;
y_log_plot = m_log*log(x_plot)+b_log;

[r2_log,rmse_log] = error(y,y_log);

figure('Name','Logarithmic Fit','NumberTitle','off');
scatter(x,y,10,'o','filled','markerfacecolor','#db70b8');
hold on;
plot(x_plot,y_log_plot,'color','#7a1f5c','LineWidth',3);
title(sprintf('Log Fit: y = %.3f ln(x) + (%.3f)\nR^2 = %.3f, RMSE = %.5f', m_log,
b_log, r2_log, rmse_log));
xlabel('WindSpeed 100m');
ylabel('Power');
legend('Data','Fit','Location','northeastoutside');
```




**Graph:**
This figure shows the original data points and the fitted logarithmic curve. This plot helps visualize if the relationship between wind speed and power appears to follow a pattern where power increases but at a decreasing rate as wind speed increases. The title includes the specific R-squared and RMSE for the Logarithmic Model.
Curve rises sharply initially, then levels off quickly. It captures the early trend where power increases with wind speed but fails to track higher values where the power continues to rise. It visually underperforms. The model flattens too early and underestimates power at high wind speeds.

**Power Model:** This model assumes a relationship of the form $\mathbf{y = b \cdot x^m}$ . By taking the logarithm of both sides, it can be transformed into a linear relationship: $\mathbf{\ln(y) = m \cdot \ln(x) + \ln(b)}$. This model is useful for relationships where y changes proportionally to a power of x.

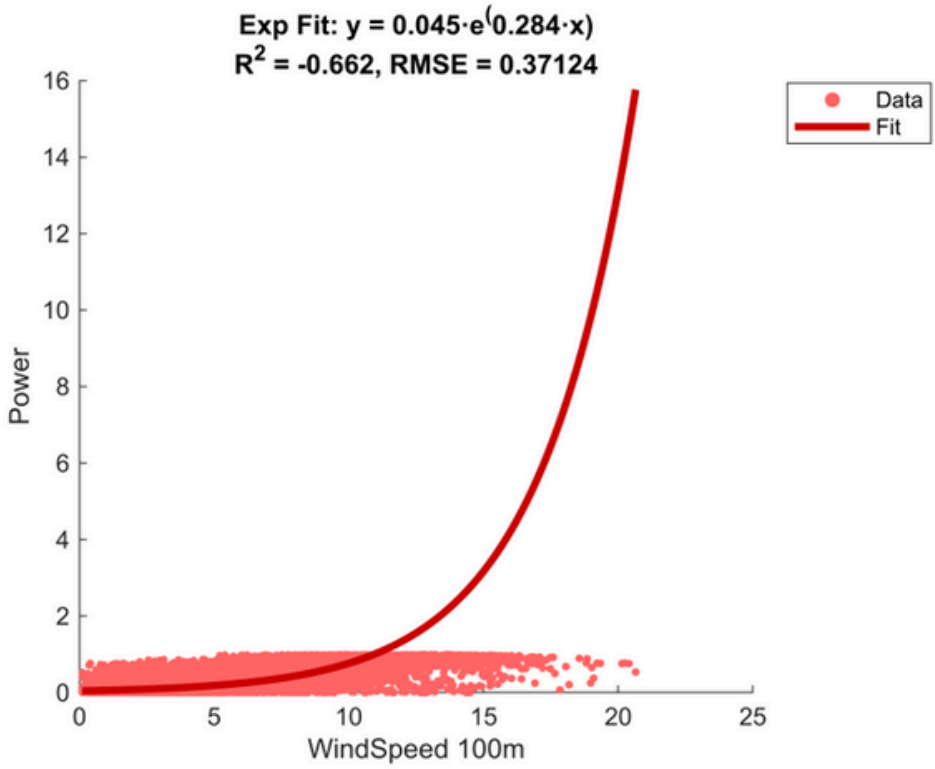
```
% -- Power Model: y = b*x^m --
lny = log(y);
p_power = polyfit(lnx,lny,1);
m_power = p_power(1);
b_power = exp(p_power(2));
y_power = b_power*x.^m_power;
y_power_plot = b_power*x_plot.^m_power;

[r2_power,rmse_power] = error(y,y_power);

figure('Name','Power Fit','NumberTitle','off');
scatter(x,y, 10,'o','filled','MarkerFaceColor','#aa80ff');
hold on;
plot(x_plot,y_power_plot,'color','#4400cc','LineWidth',3);
title(sprintf('Power Fit: y = %.3f·x^(%.3f)\nR^2 = %.3f, RMSE =
%.5f',b_power,m_power, r2_power, rmse_power));
xlabel('WindSpeed 100m');
ylabel('Power');
legend('Data','Fit','Location','northeastoutside');
```




**Graph:**
This figure displays the original data points and the fitted power law curve. This plot is useful for relationships where power is proportional to wind speed raised to some power. Wind turbine power curves often exhibit a power-law like behavior in certain wind speed ranges. The title includes the specific R-squared and RMSE for the Power Model.
A smooth curve that increases steadily with wind speed and approximates the data trend reasonably well, especially in the middle wind speed range. Overall, a good physical model with decent predictive capability, though it lags behind the degree 4 polynomial at the extremes.

**Exponential Model:** This model assumes a relationship where the dependent variable (y) changes exponentially with the independent variable (x). Its form is $\mathbf{y = b \cdot e^{m \cdot x}}$. By taking the natural logarithm of y, it can be have a linear relationship: $\mathbf{\ln(y) = m \cdot x + \ln(b)}$. This is suitable for data exhibiting exponential growth or decay.

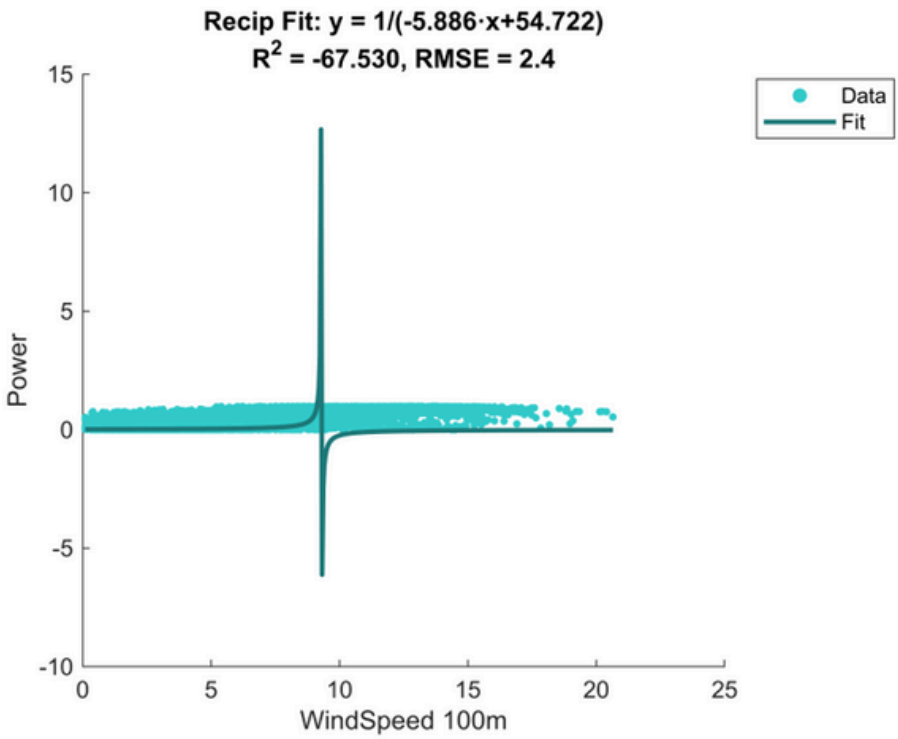
```
% -- Exponential Model: y = b*e^(m*x) --
p_exp = polyfit(x,lny,1);
m_exp = p_exp(1);
b_exp = exp(p_exp(2));
y_exp = b_exp*exp(m_exp*x);
y_exp_plot = b_exp*exp(m_exp*x_plot);

[r2_exp,rmse_exp] = error(y,y_exp);

figure('Name','Exponential Fit','NumberTitle','off');
scatter(x,y,10,'o','filled','MarkerFaceColor','#ff6666');
hold on;
plot(x_plot,y_exp_plot,'color','#cc0000','LineWidth',3);
title(sprintf('Exp Fit: y = %.3f·e^(%.3f·x)\nR^2 = %.3f, RMSE = %.5f', b_exp,
m_exp, r2_exp, rmse_exp));
xlabel('WindSpeed 100m');
ylabel('Power');
legend('Data','Fit','Location','northeastoutside');
```




**Graph:**
This figure shows the original data points and the fitted exponential curve. This plot helps visualize if the power output increases exponentially with wind speed. The title includes the specific R-squared and RMSE for the Exponential Model.
The curve starts off flat and then grows rapidly. It underpredicts power at low wind speeds and overpredicts drastically at high wind speeds. In conclusion it is visually inconsistent with actual data behavior and overestimates due to rapid exponential growth.

**Reciprocal Model:** This model assumes a relationship where the reciprocal of the dependent variable (1/y) changes linearly with the independent variable (x). Its form is **1/y = m · x+b**, which can be rearranged to **y = 1/(m · x+b)**. This model can capture relationships where y approaches an asymptote as x increases or decreases.

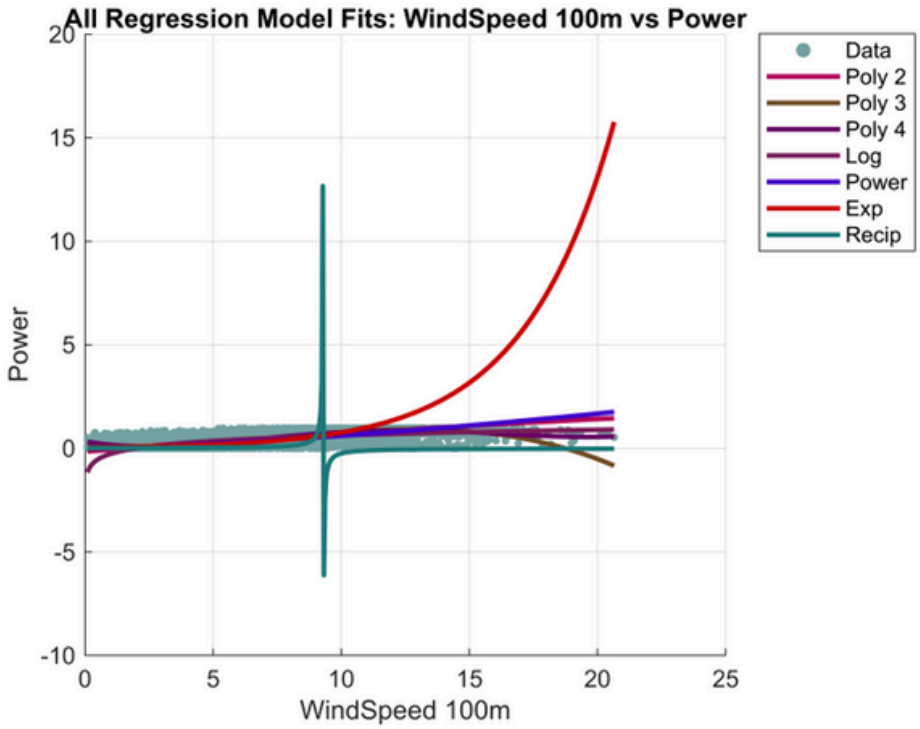
```
% -- Reciprocal Model: y = 1/(m*x+b) --
p_rec = polyfit(x,1./y,1);
m_rec = p_rec(1);
b_rec = p_rec(2);
denominator = m_rec*x+b_rec;
y_rec = 1./denominator;
denom_plot = m_rec*x_plot+b_rec;
y_rec_plot = NaN(size(x_plot));
valid_idx = abs(denom_plot) > 1e-3;
y_rec_plot(valid_idx) = 1./denom_plot(valid_idx);

[r2_rec,rmse_rec] = error(y,y_rec);

figure('Name','Reciprocal Fit','NumberTitle','off');
scatter(x,y,10,'o','filled','MarkerFaceColor','#33cccc');
hold on;
plot(x_plot,y_rec_plot,'color','#1f7a7a','LineWidth',2);
title(sprintf('Recip Fit: y = 1/(%.3f·x+%.3f)\nR^2 = %.3f, RMSE =
%.1f',m_rec,b_rec,r2_rec,rmse_rec));
xlabel('WindSpeed 100m');
ylabel('Power');
legend('Data','Fit','Location','northeastoutside');
```




**Graph:**
This figure displays the original data points and the fitted reciprocal curve. This plot can show relationships where the power output approaches a limit or changes rapidly as wind speed varies. Note that this curve has asymptotes, which appears as breaks in the fitted line on the plot. The title includes the specific R-squared and RMSE for the Reciprocal Model.
The curve behaves erratically, with sharp changes and asymptotic behavior. It does not follow the data pattern and diverges at higher wind speeds. It has poor visual performance so the model is unsuitable for this dataset.

**All Models:** This figure overlays all the successfully fitted regression curves on a single scatter plot of the original data. This combined plot allows for a direct visual comparison of how each model fits the overall trend of the data. You can see which curves seem to follow the data points most closely across the entire range of wind speeds. This graph is particularly useful for quickly assessing the relative performance of different model types and identifying which ones appear to capture the shape of the data most effectively.

```
% --- Combined Model Comparison Plot ---
figure('Name','All Models','NumberTitle','off');
scatter(x,y,10,'o','filled','MarkerFaceColor','#75a3a3');
hold on;

% Plot the polynomial fits
for i = 1:length(poly_degrees)
plot(x_plot, poly_y_plots{i}, 'Color', colors_poly{i}, 'LineWidth', 2);
end

% Plot the other models
plot(x_plot,y_log_plot,'color','#7a1f5c','LineWidth',2);
plot(x_plot,y_power_plot,'color','#4400cc','LineWidth',2);
plot(x_plot,y_exp_plot,'color','#cc0000','LineWidth',2);
plot(x_plot,y_rec_plot,'color','#1f7a7a','LineWidth',2);
xlabel('WindSpeed 100m');
ylabel('Power');
title('All Regression Model Fits: WindSpeed 100m vs Power');
legend('Data','Poly 2','Poly 3','Poly
4','Log','Power','Exp','Recip','Location','northeastoutside');
grid on;
```




**Graph:**
Degree 4 Polynomial and Power models visually align with the majority of data points while Logarithmic and exponential diverge in the higher range and Reciprocal model deviates significantly. The plot confirms that the Polynomial Degree 4 model most accurately captures the underlying trend.

**Summary Table:**

Creating a table to summarize and compare the $R^2$ and RMSE values of each model. Comparing the $R^2$ and RMSE values across the models in the Summary Table allows us to quantitatively assess their performance. A model with a higher $R^2$ and a lower RMSE is generally considered a better fit to the data.

```
% -- Summary Table --
modelNames = ["Poly 2";"Poly 3";"Poly 4"; "Logarithmic"; "Power"; "Exponential";
"Reciprocal"];
r2_values = [r2_poly(1);r2_poly(2);r2_poly(3);r2_log;r2_power;r2_exp;r2_rec];
rmse_values =
[rmse_poly(1);rmse_poly(2);rmse_poly(3);rmse_log;rmse_power;rmse_exp;rmse_rec];
summaryTable = table(modelNames,r2_values,rmse_values, ...
'VariableNames', {'Model','R_squared','RMSE'});

disp('Summary Table:');
```

```
Summary Table:
```

```
disp(summaryTable);
```

| Model | R_squared | RMSE |
|---|---|---|
| "Poly 2" | 0.61153 | 0.1795 |
| "Poly 3" | 0.65494 | 0.16917 |
| "Poly 4" | 0.66294 | 0.1672 |
| "Logarithmic" | 0.48884 | 0.2059 |
| "Power" | 0.52801 | 0.19785 |
| "Exponential" | -0.66165 | 0.37124 |
| "Reciprocal" | -67.53 | 2.3841 |

**Filtering the table:**

Only models with an R-squared value of 0 or greater were included in the summary and best model selection. Models with R-squared less than 0 or NaN were excluded, being considered unsuitable for describing the wind speed/power relationship.

```
% Filter the table to exclude models with negative R-squared
valid_r2 = summaryTable.R_squared >= 0;
filteredSummaryTable = summaryTable(valid_r2, :);

disp('Summary Table (excluding models with negative R-squared values):');
```

```
Summary Table (excluding models with negative R-squared values):
```

```
disp(filteredSummaryTable);
```

| Model | R_squared | RMSE |
|---|---|---|
| "Poly 2" | 0.61153 | 0.1795 |
| "Poly 3" | 0.65494 | 0.16917 |
| "Poly 4" | 0.66294 | 0.1672 |
| "Logarithmic" | 0.48884 | 0.2059 |
| "Power" | 0.52801 | 0.19785 |

**Best Model Selection:**
The model with the highest R-squared value among the filtered set of models was identified as the best model.

```
% Find Best Model Based on R^2 from the filtered table
[~,bestIdx]=max(r2_values);
disp('Best Model Based on R2:');
```

```
Best Model Based on R2:
```

```
disp(summaryTable(bestIdx,:));
```

| Model | R_squared | RMSE |
|---|---|---|
| "Poly 4" | 0.66294 | 0.1672 |

**Result:**
Based on the analysis of the cleaned data and the evaluation of the fitted regression models, the Polynomial model of Degree 4 provided the best fit to the data, as indicated by the highest R-squared value. This model is best for describing the relationship between wind speed at 100m and power output for this dataset.

**Conclusion:**
This study successfully applied a range of regression techniques using MATLAB to model the relationship between wind speed at 100 meters power output. A total of five nonlinear regression models—Polynomial (degrees 2–4), Logarithmic, Power, Exponential, and Reciprocal—were fitted to cleaned real-world data obtained from wind turbine operations. Each model's performance was assessed based on $R^2$ (Coefficient of Determination) and RMSE (Root Mean Square Error).
Among all models tested, the Polynomial Regression of Degree 4 emerged as the most accurate. It exhibited the highest $R^2$, capturing the complex nonlinear pattern in power generation with increasing wind speeds. Visually, the degree 4 polynomial closely followed the scatter plot of the original data, offering excellent agreement across the entire wind speed range.
Alternative models such as Power Regression showed reasonable approximation, particularly in mid-range wind speeds. However, models like Exponential, Logarithmic, and especially Reciprocal were found to be unsuitable for this application, either due to poor fit, overestimation at extremes, or numerical instability.
This analysis emphasizes the importance of careful model selection in engineering applications. Accurate modeling of wind speed–power relationships is critical for:

- Optimizing turbine performance
- Forecasting energy output
- Improving design and operational strategies in wind energy systems

From a mechanical engineering perspective, the Polynomial Degree 4 model serves as an accurate tool for turbine power prediction.

NED University of Engineering & Technology
Department of Mechanical Engineering
ME-214 Computer Programming & Applications
Rubric for the Assessment of Open-Ended Lab

| **Psychomotor Domain Assessment Rubric-Level P4** | | | | | |
|---|---|---|---|---|---|
| Skill Sets | Extent of Achievement | | | | |
| | 0 | 1 | 2 | 3 | 4 |
| **Procedural Skills**<br>***Demonstrates*** procedural skills of the open-ended lab coding. | Doesn't demonstrate procedural skills. | Slightly demonstrates procedural skills. | Somewhat demonstrates procedural skills. | Moderately demons-trates procedural skills. | Fully demonstrates procedural skills. |
| **Execution**<br>Ability to ***execute*** the program that meet all specifications | Not able to execute the program that meet all specifications. | Able to slightly execute the program that meet all specifications. | Able to somewhat execute the program that meet all specifications. | Able to moderately execute the program that meet all specifications. | Able to fully execute the program that meet all specifications. |
| **Efficiency**<br>E***fficient*** coding skills without sacrificing readability and understanding. | The code is huge and appears to be patched together. | The code is unnecessarily long. | The code is somewhat efficient without sacrificing readability and understanding. | The code is fairly efficient without sacrificing readability and understanding. | The code is extremely efficient without sacrificing readability and understanding. |
| **Documentation**<br>Able to ***show*** what the code is accomplishing and how. | There are no comments embedded in the code and does not help the reader understand the Code. | The documentation is partially comments embedded in the code and is difficult to help the reader understand the Code. | The documentation is simply comments embedded in the code with some simple header comments separating routines. | The documentation consists of embedded comment and some simple header documentation that is somewhat useful in understanding the code | The documentation is well written and clearly explains what the code is accomplishing and how. |

Laboratory Session No. _____ Date: _____

| | |
|---|---|
| Weighted CLO (Psychomotor Score) | |
| Remarks | |
| Instructor's Signature with Date: | |